

Gemma MVP

Architecture & Concepts — Deep Dive

The single source of truth: every container, every concept, in depth.

Document purpose: A complete technical reference for the 15-container Furix appliance MVP — what each container does, the engineering concepts behind it, how the production version differs, and an honest assessment of where the code and concepts stand today.

Scope: Validation MVP — proving the in-house Gemma model (`gemma4:e4b`) is accurate and fast enough to power Furix's security reasoning.

Audience: Engineering, client technical stakeholders.

Legend: PRODUCTION-GRADE CONCEPT SIMPLIFIED IN MVP ROADMAP

Contents

1. 1 · What the MVP is (and is not)
2. 2 · The five design principles
3. 3 · End-to-end data flow
4. 4 · Why you still see lag — the latency reality (in depth)
5. 5 · The 15 containers — deep dive
 - C1 nginx · C2 Vector · C3 Scan Engine · C4 Intel Sync · C5 Kafka
 - C6 Normaliser · C7 Gemma · C8 Storage+Detection · C9 Postgres · C10 ClickHouse
 - C11 Dashboard · C12 Operations · C13 Valkey · C14 AI Brain · C15 Backup
6. 6 · The DAL — privacy by construction
7. 7 · The five AI agents
8. 8 · The cost model — why the LLM is the last resort
9. 9 · How we measure (percentiles & saturation)
10. 10 · Engineering assessment — how good is each container?
11. 11 · Scope & roadmap to production
12. 12 · Glossary

1 · What the MVP is (and is not)

The Furix Gemma MVP is a **faithful, lightweight reproduction of the full 15-container Furix security appliance**, built to answer one question with evidence: *"Is our in-house Gemma model accurate enough and fast enough to power Furix's security reasoning — before we commit to the full production build?"*

✓ **What it IS**

- All **15 containers** of the real architecture, running as one Python process ("lite mode").
- All **5 AI agents** calling a real, on-prem Gemma model.
- The real **privacy layer** (DAL), **cost-routing**, and **compliance grounding**.
- A **measurement rig**: routing/cost analysis + Gemma capacity benchmarking.

✗ **What it is NOT (yet)**

- Not production-scale: single-process, in-memory by default.
- No GPU-optimised serving (vLLM), no Kafka partitioning, no horizontal workers.
- No cross-event correlation (each event reasoned independently).
- No GBDT "instinct" tier (the production middle layer between cache and LLM).

The one idea to remember: Every cheap, deterministic step runs first so that the expensive AI model is the *last resort* — invoked only when an event genuinely needs reasoning. Most of the appliance exists to make the model trustworthy (privacy + grounding + validation) and affordable (cache + deterministic routing).

The three-repo ecosystem

The MVP is the **appliance under test**. Two sibling generators produce realistic, labelled test data with known-correct answers, so the appliance's verdicts can be scored objectively.

Repo	Role
furix-gemma-mvp	The appliance: 15 containers, 5 agents, the AI Brain.
logforge	Synthetic security-log generator (22 formats, attack campaigns) + ground-truth labels.
vulnforge	Synthetic vulnerability-scan generator grounded in real CVE/KEV/EPSS + ground-truth labels.

2 · The five design principles

Every container's design follows from these. When a client asks "why is it built this way?", the answer is always one of these five.

① The LLM is the last resort

Four layers each avoid an AI call: the **verdict cache** (seen this before?), **deterministic parsing** (C6 pattern-matching), **deterministic compliance mapping** (rules + crosswalk + embeddings), and **on-demand narrative** (heavy report agents run only for serious events). The model is reached only when these can't answer.

② One bus, one knowledge graph

Logs (C2) and vulnerability scans (C3) flow through the *same* pipeline and are reasoned over the *same* knowledge graph. "A scanner that also knows your alerts, and an AI that reasons over both, beats three disconnected tools." Containers never call each other directly — they publish to the C5 bus.

③ Privacy by construction

The model never sees raw private data. C6 keeps real identifiers only long enough to join threat intel, then the **DAL** redacts everything to placeholders before any prompt, and re-hydrates only after the answer returns. The whole appliance is on-prem with a single outbound connection (threat-intel pull).

④ Deterministic and probabilistic are partners

Fast rules (C8 detection, C6 signals) catch the known-bad instantly and for free; the probabilistic model reasons about the novel and writes the human narrative. Neither replaces the other — defense in depth.

⑤ Graceful degradation

Every external dependency has an in-process fallback: memory bus instead of Kafka, ring-buffer instead of ClickHouse, TTL-dict instead of Valkey, static maps instead of the RAG graph, and `MOCK_LLM` instead of Gemma. The full 15-container pipeline runs with zero external infrastructure — which is exactly what makes the MVP portable and testable.

3 · End-to-end data flow

One log's journey, left to right. Notice the model (C7) sits at the end of a long funnel of cheaper work.

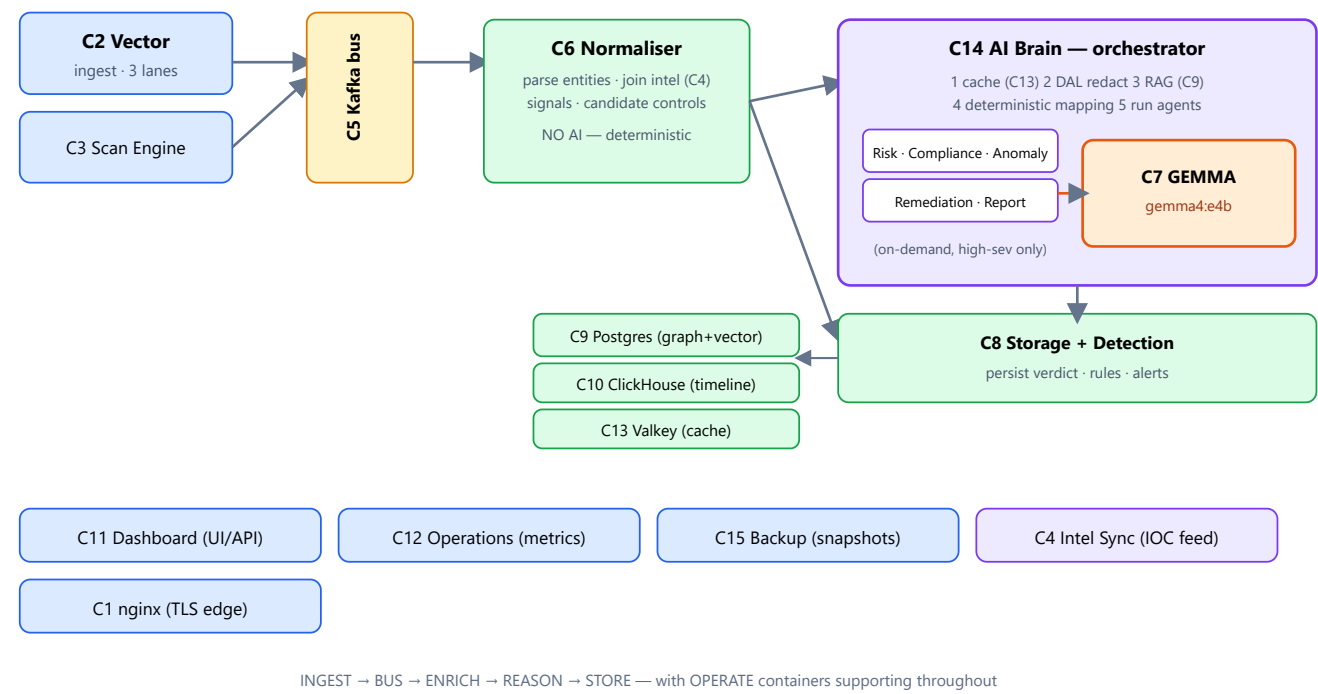


Figure 1 — The 15-container pipeline. Cheap deterministic work (green/blue) funnels into the expensive model (orange) only when necessary.

4 · Why you still see lag — the latency reality

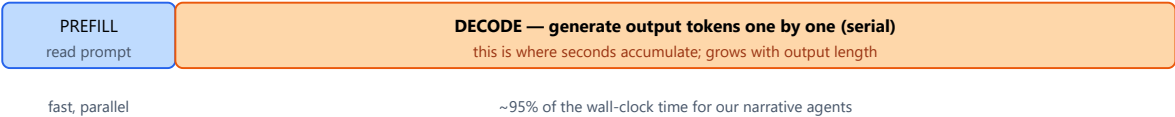
A common and fair question: "The model is on a GPU — why is it still slow?" The honest answer is that LLM latency is dominated by how language models generate text, not by whether a GPU is present. This section explains it properly.

4.1 LLMs generate one token at a time (autoregressive decoding)

An LLM does not produce its whole answer at once. It generates **one token (≈¾ of a word) at a time**, and each new token requires a *complete forward pass through the entire model*, re-reading everything generated so far. A 600-token answer = 600 sequential passes. This is inherently serial and cannot be parallelised *within a single response* — a GPU makes each pass faster, but it cannot remove the sequence.

Rule of thumb: response time ≈ `prefill` (read the prompt, fast and parallel) + `output_tokens × per-token time` (generate the answer, slow and serial). **Output length is the single biggest driver of latency.**

Where the time goes in one Gemma call



Our agents' output budgets: Risk/Anomaly ≈ short · Remediation = up to 700 tokens · Report = up to 900 tokens
→ The Report and Remediation agents are the long ones you wait for; the analytical agents are near-instant.

Figure 2 — Prefill is cheap; decode dominates. Longer answers = proportionally longer waits.

4.2 Why a GPU helps but doesn't make it instant

- **Per-token speed is bound by memory bandwidth.** Each token reads the model's weights from GPU memory. Even a fast GPU has finite bandwidth, so there is a floor on per-token time that scales with model size.
- **Concurrency ≠ single-request speed.** A GPU can serve many requests at once (throughput), but it cannot make one sequential generation finish sooner. Adding load improves total events/second, not the time for any individual answer.
- **Cold start.** The first call after idle loads the model into GPU memory — seconds to tens of seconds. We mitigate this by pre-warming and keeping the model resident.

4.3 The serving stack matters: Ollama vs vLLM

The MVP currently serves Gemma via **Ollama**, which is excellent for convenience but is optimised for single-stream use. Production targets **vLLM**, which adds:

Capability	Ollama (MVP)	vLLM (production target)
------------	--------------	--------------------------

Continuous batching (many requests share GPU passes)	Limited	Yes — large throughput gain under load
PagedAttention (efficient KV-cache memory)	No	Yes — more concurrent requests per GPU
Best for	Single user, demos, validation	Many concurrent agents/analysts

So part of the lag you see is the *serving layer*, not the model itself — moving to vLLM on GPU is a major, known lever.

4.4 The levers we have to reduce latency

Lever	Effect	Status
Don't call the model at all (cache + deterministic routing)	Eliminates the latency entirely for most events — the biggest lever	live
Run only needed agents (narrative on-demand)	Cuts ~4 calls/event to ~2 for routine events	live
Lower <code>max_tokens</code> per agent	Fewer output tokens = proportionally faster	tunable
vLLM continuous batching on GPU	10×+ concurrency; lower p95 under load	roadmap
Quantization (e.g. 4-bit) / smaller variant	Faster per-token, less memory	roadmap
GBDT "instinct" tier before the LLM	A tiny ML model answers the easy reasoning, skipping Gemma	roadmap
Keep model warm (no cold-load)	Removes first-call penalty	recommended

The architectural point: Furix's answer to LLM latency is not "buy a bigger GPU" — it is "call the model as rarely as possible, and make each call as short as possible." The Routing & Cost benchmark exists precisely to prove how often we avoid the model altogether.

5 · The 15 containers — deep dive

For each: its role, how it works in the code, the concept behind it, and how the production version differs. Maturity ratings reflect the *concept* (is the design right?) and the *MVP implementation* (how complete is the code today?).

C1 nginx — TLS edge

Role: The only door into the appliance. Terminates TLS and proxies to the dashboard (C11).

Concept: A single ingress choke point is where authentication, rate-limiting, and audit belong. Nothing else in the appliance is reachable from outside — internal containers trust the network boundary nginx enforces.

Production vs MVP: **concept** standard, battle-tested nginx config. **mvp** in lite mode the dashboard is hit directly; nginx is used in the docker-compose deployment.

C2 Vector — log ingestion & priority lanes

Role: The SIEM front door. Accepts raw events, triages each into one of three priority lanes (`HOT / WARM / COLD`), and publishes onto the bus. Throughput + routing, not understanding.

How it works: `classify_lane()` regex-matches the raw text — HOT is checked first ("a single scary keyword wins"). HOT triggers include `failed password, invalid user, mimikatz, beacon, cobalt, ransom, exploit, cve-, c2, eternalblue`. A `LaneScheduler` holds three queues drained strictly HOT→WARM→COLD. Crucially, `ingest_many()` enqueues *all* logs first, then drains once, so every HOT event is processed before any WARM — the priority guarantee only matters under a backlog.

Concept: Under a flood, a ransomware indicator must not wait behind a million routine logins. Lanes are a contention-time guarantee.

Production vs MVP: **concept** this is `vector.dev` (Rust) at 500K+ events/sec into Kafka topics `raw.HOT/WARM/COLD`. **mvp** a compact Python intake + the real lane scheduler so prioritisation is demonstrable in lite mode.

C3 Scan Engine — active vulnerability scanning

Role: The "go look" half of the platform. On demand, probes an asset, fingerprints its services, matches them against known CVEs, and emits findings onto the *same bus as logs*.

How it works: `scan(target, services)` takes service fingerprints (as nmap reports them), substring-matches a CVE catalog, and publishes each hit to `scan.findings` — so a scan finding flows through C6→C14→C8 exactly like a log. Catalog includes Zerologon (CVE-2020-1472), BlueKeep (CVE-2019-0708), Exchange EoP (CVE-2024-21410), Apache path traversal (CVE-2021-41773).

Concept: Unifying scans and logs onto one bus and one knowledge graph is the core Furix thesis — correlation between "what's vulnerable" and "what's being attacked" is impossible in three disconnected tools.

Production vs MVP: **concept** drives OpenVAS/Tenable/Qualys-class engines, 8-phase scan pipeline. **mvp** a simulated scanner with a small CVE catalog — enough to prove the unified-pipeline thesis.

C4 Intel Sync — threat intelligence

Role: Maintains a fresh cache of "known bad" indicators (malicious IPs, domains, file hashes, CVE/KEV). Other containers ask "is this known-bad?" during enrichment.

How it works: `refresh()` loads IOCs into the C13 cache keyed `ioc:{kind}:{value}`; `is_known_bad(kind, value)` is an O(1) lookup that C6 calls per extracted entity. Tracks last-sync time for health.

Concept: Intel turns "an IP connected out" into "an IP connected to a known C2 server" — the same event, a vastly different severity. The enrichment join is what creates that context.

Production vs MVP: **concept** a small Go binary that is the *only outbound flow in the entire appliance* — pulls signed feed deltas from Furix Cloud every ~4 hours over mTLS. **mvp** a built-in feed seeded into the cache.

C5 Kafka — the message bus

Role: The spine. Containers publish to named topics; subscribers consume. Decoupling lets each box scale, crash, and restart independently.

How it works: A single `publish()/subscribe()/history()` interface backed by either an in-process `_MemoryBus` (default) or a real Kafka adapter (`BUS_BACKEND=kafka`). The memory bus fans out to handlers *outside its lock*, so one slow consumer can't block publishers, and a throwing handler is caught and counted — one bad consumer never kills the bus. ~11 canonical topics: `raw.HOT/WARM/COLD`, `scan.findings`, `normalized.events`, `detection.input`, `ai.enrichment`, `ai.verdicts`, `intel.updates`, `kg.findings`, `timeline.events`.

Concept: Publishers and consumers share one contract (topic + JSON message), so the backend can swap memory↔Kafka without changing any container's logic. This is what makes "lite mode" and "production mode" the same code.

Production vs MVP: **concept** Apache Kafka in KRaft mode (no ZooKeeper), durable, replicated, at-least-once. **mvp** memory backend by default; Kafka adapter available.

C6 Normaliser — parse & enrich (no AI)

Role: Turns ~20 messy vendor formats into one canonical event. Four deterministic stages, zero LLM. The workhorse that makes data cheap for the AI Brain to reason about.

How it works: (1) regex-extract entities (IPs, domains, CVEs, usernames) and fingerprint the log type (CloudTrail, Okta, Windows EVT, Zeek, auditd...); (2) join entities against C4 intel into `ioc_hits`; (3) compute 7 behavioural **signals** (`malware`, `c2_or_exfil`, `privilege_escalation`, `account_creation`, `lateral_movement`, `failed_logins`, `successful_logins`) and map candidate CIS controls via the `KW` keyword map; (4) tag lineage and fan out to `normalized.events`, `detection.input`, and `ai.enrichment`. The same triage maps are imported by C14 as its deterministic fallback — one source of truth.

Code-quality signal: every keyword is word-boundary anchored (`\b...\b`). A documented accuracy fix: bare substrings caused catastrophic false positives — `"rce"` matched `"source"`, `"c2"` matched

"ec2.amazonaws", "s3" fired a control on benign S3 reads. This is the kind of rigor that separates a real detection engine from a demo.

Production vs MVP: **concept** Python + aiokafka, same four stages, GeolP + signature enrichment.
mvp a focused subset — the canonical event shape and triage logic are real.

C7 Gemma — local model inference (the box under test)

Role: Runs the LLM locally; 100% on-prem, no outbound calls. The AI Brain (C14) is its only client. *This is the component the entire MVP exists to validate.*

How it works: An OpenAI-compatible client (`complete_json`) calls `gemma4:e4b` . It forces strict JSON output, retries 3× with a small temperature nudge and JSON-repair (balancing truncated strings/braces) for robustness, and returns latency + token counts with each result. `health()` probes with a tiny real completion (more reliable than `/v1/models` , which some servers return null for). `MOCK_LLM=1` returns deterministic stubs for offline plumbing tests.

Concept: The model is wrapped in retries, JSON repair, and validation so a stochastic component behaves like a reliable service. The call itself is ~6 lines; the reliability scaffolding around it is the point.

Production vs MVP: **concept** vLLM serving a Gemma variant (E2B/E4B/26B-MoE/31B) with batch scheduling, 16K context, and safety rails (kill-switch, 50KB prompt cap, post-response PII scan). **mvp** single endpoint via Ollama. See §4 for latency.

C8 Storage Writer + Detection Engine

Role: Two peer jobs. (1) Storage Writer persists canonical events + AI verdicts to C9 (relational) and C10 (timeline). (2) Detection Engine runs fast deterministic rules and raises alerts.

How it works: Subscribes `normalized.events` , `detection.input` , `ai.verdicts` . Five rules: `malware_execution` (critical), `c2_or_exfil_or_ioc` (critical, fires on any IOC hit), `brute_force_success` (high = failed+successful logins together), `privilege_escalation` (high), `unauthorized_account` (high). A throwing rule is caught and counted so it can't stop the engine.

Concept: Rules and the AI are partners — rules are instant and free and catch the known; the model reasons about the novel. Note the deliberate non-duplication: C8 does *not* re-escalate to the AI Brain because C6 already routes every event there.

Production vs MVP: **concept** Go + aiokafka, idempotent batch writes, 500K rows/sec. **mvp** in-process rules + writers; same topics and shapes.

C9 Postgres + AGE + pgvector — the knowledge graph

Role: One Postgres wearing three hats — relational (findings/verdicts/audit), graph (CIS→NIST→HIPAA control relationships via Apache AGE), and vector (embedded compliance text via pgvector for RAG).

How it works: The AI Brain retrieves relevant real controls/snippets from here to *ground* the model's reasoning. A relevance floor (`RAG_SCORE_FLOOR=0.30`) drops weak matches so a novel log isn't grounded in irrelevant controls. Embeddings use a security-domain model (`SecureBERT2.0`), retrieved 35 → reranked 12 → final 6.

Concept: Three query shapes, one store — "how are these related?" (graph), "what's semantically similar?" (vector), "give me the rows" (SQL). Grounding in *your* environment is what stops the model hallucinating compliance mappings.

Production vs MVP: **concept** PostgreSQL 16 + AGE + pgvector, encrypted; the graph is the core asset. **mvp** degrades to a static compliance map when the DB isn't present (`RAG_ENABLED=0`), so the pipeline always runs.

C10 ClickHouse — columnar event timeline

Role: The "what happened, when" store — append-mostly, immutable, built for sub-second queries over billions of rows. Powers the dashboard timeline and historical search.

How it works: If `CLICKHOUSE_URL` is set it creates a `timeline_events` MergeTree table ordered by timestamp; otherwise it falls back to an in-memory ring buffer (10,000 rows) with the same `append()/recent()` surface. Columns: `ts, finding_id, log_type, severity, summary`.

Concept: Postgres answers "how are these related?"; ClickHouse answers "show every event in order, fast." Using one database for both is the classic SIEM mistake this avoids — relational and analytical workloads have opposite storage needs.

Production vs MVP: **concept** ClickHouse at 500K+ rows/sec, 10–15× compression, tiered HOT/WARM/COLD tables. **mvp** ring buffer fallback.

C11 Dashboard — the user-facing box

Role: The only container an analyst touches — verdicts, timeline, alerts, the Ops view, and the API to trigger analyses, batch ingest, scans, and backups.

How it works: A FastAPI app (the unified console with Pipeline / Routing & Cost / Gemma Capacity tabs). Endpoints include `/api/analyze`, `/api/analyze/batch`, `/api/health`, `/api/stress/*`, plus operational views.

Concept: All user input behind one box means one place for authorization, rate-limiting, and audit.

Production vs MVP: **concept** React SPA + FastAPI + WebSockets behind nginx. **mvp** FastAPI + static console — fully functional for the demo.

C12 Operations — observability

Role: The nervous system. Every container reports counters and latencies here; it exposes Prometheus metrics and a health rollup. *This is what powers the Gemma Capacity benchmark.*

How it works: `incr()` for labelled counters, `observe()` for latency samples, `latency_summary()` for p50/p95/p99/max/avg, a `timer` context manager, `render_prometheus()` emitting real Prometheus exposition format, and `health()` rolling up each container's probe (a sick probe never crashes the rollup).

Concept: Built early because you can't tune what you can't see — the same numbers shown on the capacity tab are what production would alert on.

Production vs MVP: **concept** Prometheus + Grafana + Loki + Alertmanager. **mvp** an in-process registry speaking the real Prometheus format — a real Prometheus can scrape it unchanged.

C13 Valkey — cache (the affordability lever)

Role: Fast shared key-value memory. Its highest-value job is the **verdict cache**: if C14 already reasoned about an identical finding, serve the cached verdict in microseconds instead of paying for another Gemma call.

How it works: `verdict_key(finding)` builds a stable SHA-256 key from the *shape* of a finding — `{log_type, signals, sorted(candidate_controls)}` — explicitly *not* its PII. Two structurally identical events hash to the same key, so the second is a cache hit that skips Gemma entirely. 24-hour TTL. Falls back to an in-process TTL dict when Valkey isn't present.

Concept: Caching is the single biggest reason an on-prem LLM is affordable — every hit is one expensive call you didn't make. Keying on shape (not exact text) means semantically identical events share an answer.

Production vs MVP: **concept** Valkey holding sessions, locks, the DAL re-hydration table, and fresh IOCs. **mvp** Redis client if reachable, else memory.

C14 AI Brain — the orchestrator

Role: The single decision layer between data and the model. For each finding it caches, redacts, grounds, runs the agents, and merges one verdict. The model is the *last* thing it reaches for.

How it works — six steps:

1. **Triage + redact:** normalise the RAW log (so real IPs join intel), then DAL-redact the text and the finding.
2. **Cache check (C13):** on hit, return immediately — no model call.
3. **Ground (C9 RAG):** use strong vector+graph matches, else fall back to validated static controls.
4. **Deterministic compliance mapping first:** rules + crosswalk + embeddings decide the controls; the Gemma compliance-mapper runs *only* when nothing matched and fallback is enabled.
5. **Run agents:** the 3 analytical agents in parallel; the 2 narrative agents (Remediation, Report) only for high-severity events or on explicit request.
6. **Merge + warm cache:** assemble one verdict (compliance fields from the authoritative deterministic mapping, not raw LLM output), rehydrate placeholders, and cache the result.

Concept: The orchestration around the LLM — privacy, caching, grounding, deterministic-first mapping, validation, cost-routing — is the real intellectual property. It turns a stochastic model into a trustworthy, affordable service.

Production vs MVP: **concept** async orchestrator with a three-tier router (cache → GBDT instinct → LLM) and CTG replay. **roadmap** the GBDT "instinct" tier (a tiny ML model that answers easy reasoning without the LLM) is the production middle layer not yet in the MVP.

C15 Backup Coordinator — consistent snapshots

Role: One consistent snapshot across all stateful stores at the same logical instant, so a restore yields a coherent system.

How it works: Two phases — PREPARE gathers a point-in-time view of each store; COMMIT serialises it, SHA-256 fingerprints it, writes the artifact + a manifest, then reads it back and re-hashes to verify

integrity ("trust but verify").

Concept: Backup is a *consistency* problem, not a copy problem — the hard part is freezing several independent databases at the same logical moment, and proving the artifact wasn't corrupted.

Production vs MVP: **concept** two-phase quiesce across Kafka/PG/ClickHouse/Valkey, AES-encrypted artifacts, Ed25519-signed manifests, 3-2-1 off-site fan-out. **mvp** JSON snapshot + SHA-256 manifest with integrity re-verification.

6 · The DAL — privacy by construction

The Data Abstraction Layer (part of C14) guarantees the model never sees raw private data. It is the reason "we send your logs to an AI" does not mean "we expose your data."

How it works: Two complementary strategies. (1) **Regex redaction** (`strip()`) over free text — emails, MACs, IPv6/IPv4, hostnames, secrets (`sk-...` , `AKIA...` , JWTs). (2) **Field-aware tokenization** (`tokenize()`) for values that no regex can catch — a bare username like "root", patient MRNs — based on which field they came from. Each value maps to a stable placeholder (`{IPV4_001}` , `{USER_001}`), so the same value always gets the same token. After the model answers, `rehydrate()` swaps placeholders back to real values.

Thoughtful detail: CVE IDs are *never* redacted — they're public identifiers the model needs to reason. And HIPAA Safe-Harbor patterns (SSN, phone, DOB, VIN) are *opt-in* (`DAL_HIPAA_MODE`) because they over-match security logs — a log timestamp would otherwise be redacted as a date of birth. This shows the privacy layer is tuned for real data, not just theory.

Concept: Privacy is enforced *structurally* — it's not a policy someone might forget, it's a mandatory step in the pipeline. The model physically cannot receive un-redacted PII.

7 • The five AI agents

Each agent is a single, strict, focused Gemma call with a validated JSON contract. Splitting the work into specialists (rather than one giant prompt) makes each call shorter, more accurate, and independently cacheable.

Agent	Question it answers	Cost profile
Risk Scorer	How dangerous is this? Severity + 0–100 score.	Deterministic by default (instant, no model)
Compliance Mapper	Which controls does this touch (CIS/NIST/HIPAA)?	Deterministic mapping first; model only as fallback
Anomaly Detector	Is this abnormal behaviour?	Deterministic by default (instant, no model)
Remediation Generator	What should we do about it?	Gemma call — high-severity events only
Report Generator	Write the human incident report.	Gemma call — high-severity events only

This table *is* the cost story: three of the five agents normally cost zero model time, and the two that call Gemma run only for events that warrant a full write-up. A routine login touches the model zero times.

8 · The cost model — why the LLM is the last resort

A naive AI security tool sends every event to the LLM. At enterprise log volumes that is financially and operationally impossible. Furix is built as a funnel that removes work at each stage.

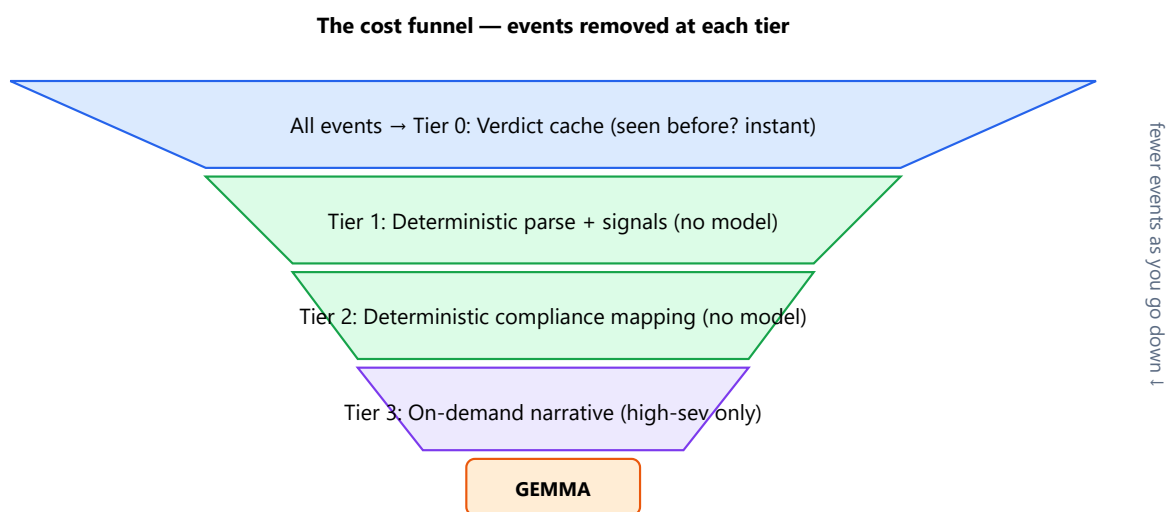


Figure 3 — Each tier answers what it can and passes only the remainder down. Only the narrow tip reaches the model.

The **Routing & Cost** benchmark in the console measures this funnel live: for every sample log it reports the route taken (deterministic vs Gemma), the model's real response time when used, and the headline KPI — *what percentage of events were resolved with no model at all*. That percentage is the cost saving, made visible.

9 · How we measure — percentiles & saturation

Why percentiles, not averages

Averages hide the painful tail. If 99 calls take 10s and one takes 5 minutes, the average looks fine while one user had a terrible experience. Percentiles expose that:

- **p50 (median)**: half of calls were faster — the typical experience.
- **p95**: 95% were faster; the slowest 5% were worse — the "bad but not rare" case.
- **p99**: 99% were faster — the worst-case tail you must plan for.

Saturation

The Gemma Capacity benchmark ramps concurrency (1, 2, 4, 8...) and watches throughput. The **saturation point** is where adding concurrency stops increasing throughput and only adds latency — i.e. the model server is full. That number is the direct input to hardware sizing: keep concurrency at or below where p95 stays acceptable.

Because the sibling generators (logforge/vulnforge) ship ground-truth labels, accuracy can be scored objectively too — precision/recall of threat detection and compliance mapping — not just speed.

10 · Engineering assessment — how good is each container?

An honest read on code quality and concept maturity. "Concept" = is the design the right one for production? "MVP code" = how complete is today's implementation?

Container	Concept	MVP code	Notes
C2 Vector	★★★★★	★★★★★	Lane scheduler is real and demonstrable; bulk-then-drain logic is correct.
C3 Scan Engine	★★★★★	★★★★	Unified-pipeline thesis proven; CVE catalog is small (by design).
C4 Intel Sync	★★★★★	★★★★★	O(1) IOC join is the right primitive; feed is seeded, not live.
C5 Kafka bus	★★★★★	★★★★★	Clean dual-backend abstraction; fan-out-outside-lock is production-minded.
C6 Normaliser	★★★★★	★★★★★	Word-boundary fix shows real detection-engineering maturity.
C7 Gemma client	★★★★★	★★★★★	Retries + JSON repair are robust; serving layer (Ollama→vLLM) is the upgrade.
C8 Storage+Detect	★★★★★	★★★★★	Rules + AI partnership is right; no-double-escalate is a thoughtful detail.
C9 Postgres KG	★★★★★	★★★★	Tri-modal store is the core asset; runs degraded (static map) in lite mode.
C10 ClickHouse	★★★★★	★★★★★	Right tool for timeline; ring-buffer fallback keeps the demo running.
C11 Dashboard	★★★★★	★★★★★	One choke point is correct; production adds SPA + WebSockets.
C12 Operations	★★★★★	★★★★★	Real Prometheus format; genuinely usable observability.
C13 Valkey cache	★★★★★	★★★★★	Shape-based key is the clever bit; the affordability lever works as designed.
C14 AI Brain	★★★★★	★★★★★	The IP. Six-step orchestration is real; GBDT instinct tier is roadmap.
C15 Backup	★★★★★	★★★★★	Consistency-first framing + integrity re-verify is correct; crypto/off-site is roadmap.

Overall: The *concepts* are production-grade across the board — this is a genuine enterprise architecture, not a toy. The *MVP code* faithfully implements those concepts with deliberate simplifications (in-memory stores, seeded feeds, Ollama serving) that exist to make the appliance portable and to focus effort on the one question that matters now: is Gemma good enough?

11 · Scope & roadmap to production

Area	MVP today	Production target
Model serving	Ollama, single-stream	vLLM on GPU — continuous batching, PagedAttention
Reasoning tiers	cache → LLM	cache → GBDT instinct → LLM (skip the model for easy cases)
Correlation	each event independent	cross-event correlation (multi-step attack campaigns reasoned as one story)
Scale	single process, in-memory	Kafka partitioning + parallel workers; real PG/ClickHouse/Valkey
Data	synthetic (logforge/vulnforge)	validated against real Tenable/Nessus + real logs
Backup	local JSON + SHA-256	AES-encrypted, signed manifests, 3-2-1 off-site

The next milestone is the real-Gemma benchmark (accuracy + capacity numbers for `gemma4:e4b`), followed by vulnforge ingestion and cross-log correlation — the highest-impact detection-quality upgrade.

12 · Glossary

Term	Meaning
Autoregressive decoding	How LLMs generate text — one token at a time, each requiring a full model pass. The reason output length drives latency.
Prefill / Decode	The two phases of an LLM call: reading the prompt (fast, parallel) vs generating the answer (slow, serial).
Token	A chunk of text ($\sim\frac{3}{4}$ of a word) — the unit LLMs read and generate in.
p50 / p95 / p99	Latency percentiles — the value under which 50% / 95% / 99% of calls complete. Honest alternatives to the average.
Saturation point	The concurrency beyond which more load adds latency, not throughput — the server is full.
IOC	Indicator of Compromise — a known-bad IP, domain, or file hash.
KEV	Known Exploited Vulnerability — a CVE confirmed to be exploited in the wild (prioritised).
RAG	Retrieval-Augmented Generation — grounding the model with retrieved real knowledge to prevent hallucination.
DAL	Data Abstraction Layer — the privacy layer that redacts PII before the model and rehydrates after.
Verdict cache	Cached AI answers keyed by event shape; a hit skips all model calls.
GBDT instinct tier	Roadmap: a small gradient-boosted model between cache and LLM that answers easy reasoning cheaply.
vLLM / Ollama	LLM serving engines. Ollama = convenient/single-stream (MVP); vLLM = high-throughput batching (production).

Furix Gemma MVP — Architecture & Concepts Deep Dive. Generated as the single source of truth for the appliance. For the live system, see the console (Pipeline · Routing & Cost · Gemma Capacity).